

OOP – Verantwoordelijkheid, vragen stellen en taken delegeren

Bij deze opdrachten draait het niet alleen om code die “werkt”. Het belangrijkste is dat iedere class een duidelijke verantwoordelijkheid krijgt.

Denk bij iedere class steeds aan drie vragen:

- Welke informatie weet deze class zelf?
- Welke vragen kan deze class beantwoorden en welke taken kan deze class uitvoeren?
- Welke andere objecten heeft deze class nodig om vragen of taken aan te delegeren?

Algemene regels

Voor alle opdrachten gelden deze regels:

- Alle klassevariabelen zijn **private**.
- Primitive data, **String**, en andere eenvoudige waarden mogen niet direct van buitenaf benaderd worden.
- Gebruik getters en waar nodig setters.
- Voeg zelf object-typed klassevariabelen toe wanneer een class een andere class nodig heeft.
- Methoden mogen taken delegeren aan andere objecten.
- Probeer te voorkomen dat één class de verantwoordelijkheden van een andere class overneemt.
- Gebruik geen **Scanner** of gebruikersinvoer tenzij dit expliciet gevraagd wordt.
- Test je programma vanuit een **main**-methode met zelfgekozen objecten en waarden.

Opdracht 1 – Bioscoop

Een bioscoop verkoopt kaartjes voor films.

Je krijgt de volgende classes en data:

```
class Movie {
    private String title;
    private int durationMinutes;
    private double ticketPrice;
}
```

```
class Ticket {
    private int seatNumber;
    private boolean used;
}
```

```
class Cinema {
    private String name;
}
```

Bedenk zelf welke objecten deze classes van elkaar moeten kennen.

Het programma moet de volgende vragen en taken aankunnen:

- Aan een **Movie** moet gevraagd kunnen worden wat een kaartje kost.
- Aan een **Ticket** moet gevraagd kunnen worden voor welke film het kaartje is.
- Aan een **Ticket** moet gevraagd kunnen worden hoeveel het kaartje heeft gekost.
- Een **Ticket** moet gebruikt kunnen worden.
- Een al gebruikt ticket mag niet opnieuw gebruikt worden.
- Aan een **Cinema** moet gevraagd kunnen worden hoeveel geld met één bepaald ticket verdiend is.

Maak minimaal passende constructors, getters/setters en de benodigde methoden.

Voorbeelden van mogelijke vragen:

```
ticket.getPrice();
ticket.isUsed();
ticket.use();
```

Bedenk vooral: wie weet de prijs werkelijk?

Opdracht 2 – Huisdier en eigenaar

Je krijgt:

```
class Owner {
    private String name;
    private int age;
}
```

```
class Pet {
    private String name;
    private int age;
    private double weight;
}
```

```
class VeterinaryClinic {
    private String clinicName;
}
```

Voeg zelf de benodigde object-relaties toe.

Het programma moet het volgende kunnen:

- Aan een huisdier moet gevraagd kunnen worden wie zijn eigenaar is.
- Aan een huisdier moet gevraagd kunnen worden of zijn eigenaar volwassen is.
- Een eigenaar is volwassen wanneer zijn leeftijd minimaal 18 is.
- Een dierenkliniek moet kunnen bepalen of een huisdier een volwassen eigenaar heeft.
- Een dierenkliniek moet de naam van de eigenaar van een huisdier kunnen opvragen.

Denk goed na over deze vraag:

Als de kliniek wil weten of de eigenaar volwassen is, moet de kliniek dan zelf de leeftijd van de eigenaar controleren?

Of kan die vraag gedelegeerd worden?

Opdracht 3 – Game character

Je krijgt:

```
class Weapon {
    private String name;
    private int damage;
}
```

```
class Player {
    private String name;
    private int health;
}
```

```
class Enemy {
    private String name;
    private int health;
}
```

Bepaal zelf welke object-typed klassevariabelen nodig zijn.

Het programma moet het volgende kunnen:

- Een speler kan een vijand aanvallen.
- De hoeveelheid schade wordt bepaald door het wapen van de speler.
- Een vijand kan schade ontvangen.
- Health mag niet lager worden dan **0**.
- Aan een vijand kan gevraagd worden of hij verslagen is.
- Een speler moet aan zijn wapen kunnen vragen hoeveel schade het veroorzaakt.

Voorbeelden van mogelijke methoden:

```
player.attack(enemy);
enemy.takeDamage(amount);
enemy.isDefeated();
weapon.getDamage();
```

Let op de verantwoordelijkheden.

Player geeft de opdracht om aan te vallen.

Maar wie weet hoeveel damage een wapen doet?

En wie is verantwoordelijk voor het aanpassen van de health van een **Enemy**?

Opdracht 4 – Restaurant

Je krijgt:

```
class Dish {
    private String name;
    private double price;
}
```

```
class OrderItem {
    private int quantity;
}
```

```
class Order {
    private int orderNumber;
}
```

```
class Restaurant {
    private String name;
}
```

Bepaal zelf de relaties tussen de classes.

Een bestelling kan uit meerdere **OrderItem**-objecten bestaan.

Het programma moet het volgende kunnen:

- Een **OrderItem** hoort bij één gerecht.
- Aan een **OrderItem** moet gevraagd kunnen worden wat zijn totale prijs is.
- De totale prijs van een **OrderItem** is:

```
aantal * prijs van het gerecht
```

- Aan een **Order** moet gevraagd kunnen worden wat de totale prijs van de hele bestelling is.
- Een restaurant moet de totale waarde van een bestelling kunnen opvragen.

Voorbeeld:

```
Pizza      €12,00 × 2 = €24,00
Cola       €3,00 × 3 =  €9,00
Totaal:    €33,00
```

Belangrijk: zorg ervoor dat **Order** niet zelf de prijs van ieder gerecht hoeft uit te rekenen als een **OrderItem** die vraag al kan beantwoorden.

Opdracht 5 – Schoolresultaten

Je krijgt:

```
class Student {
    private String name;
    private int studentNumber;
}
```

```
class Grade {
    private double value;
}
```

```
class Course {
    private String name;
}
```

```
class School {
    private String name;
}
```

Een student kan meerdere cijfers hebben.

Een cijfer hoort bij één vak.

Bepaal zelf welke classes elkaar moeten kennen.

Het programma moet de volgende vragen kunnen beantwoorden:

- Voor welk vak is een bepaald cijfer?
- Is een cijfer voldoende?
- Een cijfer is voldoende wanneer het minimaal **5.5** is.
- Wat is het gemiddelde cijfer van een student?
- Hoeveel voldoende heeft een student?
- Heeft een student een voldoende voor een bepaald cijfer?
- Een school moet het gemiddelde cijfer van een student kunnen opvragen.

Voorbeeld:

```
Programmen: 7.4
Database:   5.2
Engels:     6.8
Gemiddelde: 6.47
Aantal voldoende: 2
```

Denk goed na over de vraag:

Wie bepaalt of een **Grade** voldoende is?

Opdracht 6 – Hotel

Vanaf deze opdracht krijg je niet meer precies voorgeschreven welke data in welke class moet staan.

Maak minimaal de volgende classes:

```
Hotel
Room
Guest
Booking
```

Het systeem moet informatie kunnen opslaan over:

- naam van het hotel;
- kamernummer;
- prijs per nacht;
- naam van een gast;
- leeftijd van een gast;
- aantal geboekte nachten;
- of een kamer beschikbaar is.

Bepaal zelf:

- welke class eigenaar wordt van iedere waarde;
- welke primitive/String-klassevariabelen iedere class krijgt;
- welke object-typed klassevariabelen nodig zijn.

Het programma moet daarna het volgende kunnen:

- Aan een kamer vragen wat één nacht kost.
- Aan een gast vragen of hij/zij volwassen is.
- Aan een boeking vragen hoeveel de boeking in totaal kost.
- Aan een boeking vragen wie de gast is.
- Aan een klant vragen welke kamer geboekt is.
- Een boeking uitvoeren.
- Bij het boeken moet de kamer niet meer beschikbaar zijn.
- Een hotel moet kunnen vragen hoeveel een bepaalde boeking kost.

Berekening:

```
totaal prijs = prijs per nacht × aantal nachten
```

Voorbeeld:

```
Kamer: 204
Prijs per nacht: €85
Aantal nachten: 3
Totaal: €255
```

Opdracht 7 – Webshop

Maak minimaal:

```
Product
ShoppingCartItem
ShoppingCart
Customer
```

Je hebt de volgende informatie nodig:

- productnaam;
- productprijs;
- voorraad;
- aantal exemplaren van een product in een winkelwagen;
- naam van een klant;
- leeftijd van een klant.

Bepaal zelf waar iedere waarde thuishoort.

Het programma moet het volgende kunnen:

- Aan een product vragen wat het kost.
- Aan een product vragen of een bepaald aantal exemplaren op voorraad is.
- Aan een winkelwagenregel vragen hoeveel die regel totaal kost.
- Aan de winkelwagen vragen wat de totale prijs is.
- Aan de winkelwagen vragen hoeveel verschillende regels erin staan.
- Aan een klant vragen of hij/zij volwassen is.
- Een product aan de winkelwagen toevoegen met een bepaald aantal exemplaren.

Voorbeeld:

```
2 × Keyboard à €40 = €80
3 × Mouse à €15   = €45
Totaal = €125
```

Extra denkpunt:

Stel dat je wilt controleren:

```
Zijn er 4 keyboards beschikbaar?
```

Welke class hoort deze vraag te beantwoorden?

Opdracht 8 – Bibliotheek

Maak een klein bibliotheeksysteem met minimaal:

```
Library
Book
Member
Loan
```

Je moet gegevens kunnen opslaan over:

- titel van een boek;
- auteur;
- of een boek beschikbaar is;
- naam van een lid;
- lidnummer;
- leeftijd van een lid;
- aantal dagen dat een boek geleend wordt.

Bepaal zelf welke class welke data bezit.

Het programma moet kunnen:

- Aan een boek vragen of het beschikbaar is.
- Een boek uitleenen.
- Een boek terugbrengen.
- Aan een lening vragen welk boek geleend is.
- Aan een lening vragen door welk lid het boek geleend is.
- Aan een lid vragen of het een volwassene is.
- Aan een lening vragen of de uitleent termijn te lang is.

Een boek mag maximaal 21 dagen geleend worden.

Belangrijk:

Wanneer een boek wordt uitgeleend, moet zijn beschikbaarheid veranderen.

Bedenk wie de taak:

```
"markeer dit boek als uitgeleend"
```

zou moeten uitvoeren.

Opdracht 9 – Bezorgdienst

Ontwerp een systeem voor een bezorgdienst.

Gebruik minimaal:

```
DeliveryService
Delivery
Package
Address
```

Je wilt onder andere informatie bewaren over:

- gewicht van een pakket;
- straat;
- huisnummer;
- plaats;
- afstand van de bezorging in kilometers;
- basisprijs van een bezorging.

Bepaal zelf waar deze data hoort te staan.

De prijs van een bezorging wordt berekend met:

```
basisprijs + afstand × €8,50
```

Wanneer een pakket zwaarder is dan 10 kg komt daar €5 toeslag bij.

Het programma moet kunnen:

- Aan een pakket vragen of het zwaar is.
- Aan een adres vragen om een volledige adresregel.
- Aan een bezorging vragen waar deze naartoe gaat.
- Aan een bezorging vragen hoeveel deze kost.
- Aan de bezorgdienst vragen hoeveel een bepaalde bezorging kost.

Voorbeeld:

```
Basisprijs:      €4,00
Afstand:         20 km
Afstandskosten: €10,00
Zwaar pakket:    €5,00
-----
Totaal:          €19,00
```

Denk hierbij vooral na over de toeslag.

Om te bepalen of de toeslag nodig is, moet **Delivery** dan zelf het gewicht controleren?

Of kan **Delivery** een vraag stellen aan **Package**?

Opdracht 10 – Pretpark

Ontwerp een systeem voor een pretpark.

Gebruik minimaal:

```
ThemePark
Visitor
Attraction
Ticket
```

Het systeem moet onder andere informatie bevatten over:

- naam van het pretpark;
- naam van een bezoeker;
- leeftijd van een bezoeker;
- naam van een attractie;
- minimale leeftijd voor een attractie;
- prijs van een ticket;
- of een ticket gebruikt is.

Bepaal zelf welke class verantwoordelijk is voor iedere waarde.

Het programma moet het volgende kunnen:

- Aan een bezoeker vragen hoe oud die is.
- Aan een attractie vragen of een bepaalde bezoeker oud genoeg is.
- Aan een ticket vragen voor welke bezoeker het is.
- Aan een ticket vragen wat het heeft gekost.
- Een ticket gebruiken om een attractie te bezoeken.
- Een bezoeker die te jong is, mag de attractie niet betreden.
- Een pretpark moet een bezoek kunnen laten uitvoeren.

Een mogelijke opdracht vanuit **main** zou uiteindelijk ongeveer deze betekenis moeten hebben:

```
Pretpark, laat deze bezoeker met dit ticket deze attractie bezoeken.
```

Het pretpark hoeft daarbij niet noodzakelijk alles zelf te controleren.

Bedenk welke vragen het kan delegeren:

```
Ticket, ben jij nog geldig?
Ticket, voor welke bezoeker ben jij?
Attractie, mag deze bezoeker naar binnen?
```

En welke taken daarna uitgevoerd moeten worden.

Bij iedere opdracht

Lever naast je Java-code ook een klein verantwoordelijkheidsverzicht in.

Gebruik bijvoorbeeld:

```
Class: Order
Weet zelf:
- orderNumber
Kent:
- meerdere OrderItem-objecten
Kan vragen beantwoorden:
- Wat kost deze bestelling?
- Hoeveel orderregels bevat deze bestelling?
Kan taken uitvoeren:
- Een OrderItem toevoegen
Delegeert:
- Vraagt aan iedere OrderItem wat deze kost.
```

Maak zo'n overzicht voor iedere belangrijke class.

Controleer daarna je ontwerp met deze vraag:

Als een class informatie nodig heeft, weet hij die informatie zelf, of weet hij aan welk object hij die vraag kan stellen?

En bij een taak:

Moet deze class dit echt zelf doen, of hoort deze verantwoordelijkheid bij één van zijn ondergeschikte objecten?